

Advanced Multi-Function Web Technology Tasks for Engineer's Day

Here are five advanced, multi-function tasks designed for the Web Technologies event on Engineer's Day. Each task covers frontend, backend, performance, and security aspects, giving students a real-world development challenge.

Task 1 — Full-Stack Event Management Product (Multi-feature Portal)

Objective: Build a production-style event portal where admins create events and students register, pay (simulated), and manage registrations.

Mandatory features (must):

- User roles: Admin & Participant (role-based routing + protected endpoints)
- Authentication + authorization (JWT or OAuth flow)
- Event CRUD (create/edit/delete events) with rich text description and images
- Registration flow with simulated payment (mock gateway or Stripe test mode)
- Search & filter events (date, category, location) + pagination
- Admin analytics page: registrations, conversion rate, daily signups chart
- File upload for attachments (e.g., ID or resume) with size/type validation

Bonus (extra points):

- Email notifications
- Calendar export (.ics)
- Role-based dashboard customization
- Dockerized app

Suggested stack: React/Next.js or Angular; Node.js/Express or Spring Boot; PostgreSQL or MongoDB; JWT; Docker.

Deliverables: GitHub repo, README, deployed demo, 3-min demo video, architecture.md.

Scoring (100 pts): Functionality 40, Auth/Role security 15, UI/UX 15, Search/Filters 10, Analytics 10, Docs 10.

Task 2 — Real-time Collaborative Code Playground

Objective: Create a collaborative code editor where multiple users edit code in real time, run it safely, and communicate.

Mandatory features:

- Real-time multi-user editing (WebSocket / socket.io)

- Syntax highlighting & language mode
- Run/execute code in a sandboxed environment
- Save/load code snippets with versioning
- In-editor chat and presence indicators

Bonus: Collaborative cursors, snippet permissions, linting/test runner integration.

Suggested stack: React + Monaco Editor, Node.js + socket.io, Redis for presence, Dockerized runner.

Deliverables: Repo, live demo, sandbox instructions, test accounts.

Scoring (100 pts): Real-time sync 30, Editor UX 20, Safe execution 20, Persistence 10, Chat 10, Docs 10.

Task 3 — Offline-First PWA: Notes with End-to-End Sync & Conflict Resolution

Objective: Build a PWA that works offline, syncs to server when online, and handles conflicts.

Mandatory features:

- Installable PWA with manifest + service worker
- Local IndexedDB storage for notes
- Background sync
- Conflict detection + resolution UX
- Full-text search across notes

Bonus: End-to-end encryption, push notifications, multi-device sync.

Suggested stack: React PWA + Workbox, Node/Express, IndexedDB.

Deliverables: Repo, deployed PWA, offline/online demo, README with design.

Scoring (100 pts): Offline 30, Sync/conflicts 25, PWA setup 15, Search 10, Security 10, Docs 10.

Task 4 — High-Performance Searchable E-Catalog (SSR + Client Perf)

Objective: Build a fast searchable product/catalog SPA with SSR, image optimization, and performance budget.

Mandatory features:

- Server-Side Rendering (SSR)
- Full search with filters
- Efficient image handling (responsive, lazy load)

- Accessibility compliance
- Client caching & infinite scroll

Bonus: Headless CMS integration, server-side caching, Lighthouse >85.

Suggested stack: Next.js/Nuxt, Node API, Elasticsearch, Cloudinary.

Deliverables: Repo, deployed SSR demo, Lighthouse report, README.

Scoring (100 pts): SSR 20, Search 25, Performance 20, Accessibility 15, UX 10, Docs 10.

Task 5 — Security & Optimization: Harden a Vulnerable Web App

Objective: Fix vulnerabilities in a web app and optimize performance.

Mandatory fixes/features:

- Fix XSS, CSRF, SQL Injection/NoSQL Injection, IDOR, auth flaws
- Add CSP, secure cookies, input validation, rate limiting
- Add tests proving vulnerabilities fixed
- Optimize performance (minify, code splitting, gzip, caching)

Bonus: CI pipeline with linting & security scan, security report.

Suggested stack: Any backend, Docker container with vulnerabilities.

Deliverables: Forked repo with fixes, test suite, before/after reports, deployment instructions.

Scoring (100 pts): Fixes 40, Security hardening 20, Performance 20, CI 10, Docs 10.

Common Deliverables (for all tasks)

- Public GitHub repo with README
- Demo video (2–4 min)
- Working deployment link
- Architecture diagram + list of libraries/APIs used

Judging Rubric & Logistics

Functionality & correctness (40%)

Code quality & tests (15%)

UI/UX & responsiveness (15%)

Performance & security (15%)

Documentation & demo (15%)

Tie-breakers: innovation, best architecture explanation, fastest load time.